System Architecture and Technical Design Document

Smart Energy-Driven Irrigation for Agricultural Value Chains (Agri-Intelligence 2026)

Document Control

Field	Value
Document title	System Architecture and Technical Design Document
Project	Smart Energy-Driven Irrigation for Agricultural Value Chains — Agri-Intelligence 2026
Site	Norton, Zimbabwe (17°49'47.9"S, 31°01'39.8"E)
Version	1.0
Date	2026-09-09
Status	Living document — reflects the codebase as of this date
Prepared from	Direct inspection of `backend/`, `irrigation-dashboard/`, `mobile-app/`, `firmware/`, `render.yaml`, `PROJECT-OBJECTIVES.md`, `PROJECT_CONTEXT.md`, `CHANGED_FILES.txt`
Companion documents	`SOFTWARE_USED.txt`, `SYSTEM_ARCHITECTURE.md` (prior, narrower revision), `CHANGED_FILES.txt`
Diagram assets	`architecture-diagrams/01-…` through `14-…` (see §22)

Revision note. This document supersedes the informal architecture notes in `SYSTEM_ARCHITECTURE.md` (2026-09-09, earlier same day) with a complete, validated specification against every subsystem requested, including subsystems (energy hardware, GIS, database) that do **not** yet exist in code and are documented here as planned/absent rather than assumed.

Table of Contents

1. Introduction
2. System Overview
3. System Requirements
4. Overall System Architecture
5. Hardware Architecture

6. Software Architecture
7. Communication Architecture
8. Data Flow Architecture
9. Irrigation Control Architecture
10. Energy Management Architecture
11. AI Architecture
12. GIS and Network Monitoring
13. Alert and Notification Architecture
14. API Architecture
15. Database Architecture
16. Security Architecture
17. Deployment Architecture
18. System Sequence Diagrams
19. Technology Stack
20. Hardware/Software Interface Table
21. Testing and Validation
22. Diagram Index
23. Architecture Validation and Implementation Gaps
24. Scalability
25. Risks and Limitations
26. Future Improvements
27. Conclusion
28. Executive Architecture Summary

List of Figures

- Figure 1 — Overall System Architecture (§4)
- Figure 2 — Hardware Architecture (§5)
- Figure 3 — Software Architecture Layers (§6)
- Figure 4 — Communication Architecture (§7)
- Figure 5 — Data Flow Diagram (§8)
- Figure 6 — Irrigation Control Architecture (§9)
- Figure 7 — Energy Management Architecture (§10)
- Figure 8 — AI Architecture (§11)
- Figure 9 — Alert and Notification Architecture (§13)
- Figure 10 — GIS / Farm Map Architecture (§12)
- Figure 11 — Database / Data Model (Entity Diagram) (§15)
- Figure 12 — API Architecture (§14)
- Figure 13 — Deployment Architecture (§17)
- Figure 14 — Security Architecture (§16)
- Figures 15—21 — Sequence Diagrams A—G (§18)

List of Tables

- Table 1 — Functional Requirements (§3)
- Table 2 — Non-Functional Requirements (§3)
- Table 3 — Hardware Requirements (§3)
- Table 4 — Software Requirements (§3)
- Table 5 — Network Requirements (§3)
- Table 6 — Security Requirements (§3)
- Table 7 — Bed / Crop Data Model (§15)
- Table 8 — Technology Stack (§19)
- Table 9 — Hardware/Software Interface Table (§20)
- Table 10 — Implementation Status Matrix (§23)

---

1. Introduction

This document specifies the system architecture of the Smart Energy-Driven Irrigation system built for the Agri-Intelligence 2026 exhibition — a solar-aware, per-bed drip irrigation platform for a small farm in Norton, Zimbabwe. It is written directly from the project's source code, configuration, and firmware, not from the original proposal, so that every diagram and claim in this document can be traced back to a specific file. Where the original design intent (captured in `PROJECT-OBJECTIVES.md`) has not yet been implemented, this document says so explicitly rather than presenting the intended design as built.

The system's actual current scope, stated plainly: three client-facing applications (a web dashboard, a mobile app, and one physical control node) share a single backend API and a single identity provider. Of the three beds the system manages, **one** (Bed 1) has real, physically wired hardware behind it — an ESP32 driving a 12V pump and solenoid valve, triggered manually over HTTP. The other two beds, and every energy/solar/sensor reading the dashboards display, are backend-generated simulation data. This is not a criticism of the project — it is an accurate description of an exhibition-stage MVP, and this document treats it as such throughout.

2. System Overview

```
FARMER  →  WEB DASHBOARD / MOBILE APP  →  BACKEND API  →  (in-memory + JSON-file state)
                                                ↓
                                     GEMINI AI · TWILIO WHATSAPP
                                                ↓
                                   ESP32 (BED 1) via HTTP, LAN or ngrok tunnel
                                                ↓
                                      PUMP + SOLENOID VALVE (BED 1 only)
```

Three software deliverables, one hardware node:

1. **`irrigation-dashboard/`** — React/Vite web dashboard (kiosk-oriented), brand "Goho Rakangwara".
2. **`mobile-app/`** — Expo/React Native app, brand "Agri-Intelligence".
3. **`backend/`** — one FastAPI service both clients call.
4. **`firmware/sensor-node/bed1-manual-irrigation/`** — the one physical ESP32 node that exists.

3. System Requirements

Table 1 — Functional Requirements

ID	Requirement	Status
FR-1	Display live status (moisture, state, last irrigation) for 3 beds	Implemented (simulated data for all 3)
FR-2	Allow a user to manually trigger irrigation on a bed	Implemented
FR-3	Actuate a real pump + valve on manual trigger for at least one bed	Implemented (Bed 1 only)
FR-4	Automatically irrigate a bed when its moisture crosses a threshold, no human action	Planned, not implemented
FR-5	Authenticate users before granting API access	Implemented (Supabase JWT)
FR-6	Provide an AI conversational assistant grounded in system state	Implemented (Gemini)
FR-7	Diagnose crop health from a photo	Implemented (Gemini vision)
FR-8	Send WhatsApp alerts on irrigation actions	Implemented (fires on trigger/start/stop)
FR-9	Send WhatsApp alerts on sensor/system fault conditions (low battery, pump fault, sensor offline, etc.)	Planned — settings model has toggles for these events; no detection logic exists
FR-10	Display farm location and device positions on a map	Implemented (static farm boundary + markers), not live device telemetry
FR-11	Persist configuration across restarts	Implemented (JSON file)
FR-12	Persist historical sensor/energy readings across restarts	Not implemented — no database
FR-13	Allow remote (off-farm-LAN) control of Bed 1's real hardware	Implemented (Render deployment + ngrok tunnel + shared secret)

Table 2 — Non-Functional Requirements

ID	Requirement	Status
NFR-1	API responses authenticated on every request	Implemented
NFR-2	Backend recoverable from a crash without data loss	Not met — in-memory bed/activity state resets on restart
NFR-3	Dashboard usable unattended on a kiosk display	Implemented (Chromium kiosk target)
NFR-4	Mobile app usable when the backend is briefly unreachable	Implemented (AsyncStorage cache + offline banner)
NFR-5	Real hardware calls fail safely (no false "success" state)	Implemented — `call_esp32()` raises before any state mutation
NFR-6	System usable by a farmer with no technical background	Partially — settings/maintenance pages assume some familiarity

Table 3 — Hardware Requirements

Component	Status
ESP32 DevKit (Bed 1)	Present, wired, in service
2N2222 transistor + 1N4007 diode + 220Ω resistor (pump relay pilot)	Present, wired
12V/30A automotive relay (pump)	Present, wired
2-channel relay module (valve)	Present, wired
DC submersible pump (STURDY ZQB-12V, 180W)	Present, wired
Bed 1 solenoid valve	Present, wired
Resistive soil-moisture probes (Beds 1–3)	**Not present** — `PROJECT-OBJECTIVES.md` §1 lists these as designed but "not yet built"
Flow / pressure / tank-level / climate sensors	**Not present** — only referenced as static mock rows in the backend's `SENSORS` list
50W solar panel + 12V battery	Documented as on hand; no telemetry wired to any software
ATS (Automatic Transfer Switch)	Documented as "simulated via a 12V adapter" for live demo — not an electronic/software-controlled ATS
Additional ESP32 nodes (2, 3)	**Not present** — the backend's `ESP32_NODES` list is static demo data, not live heartbeats

Table 4 — Software Requirements

Layer	Requirement
Backend	Python 3.x, FastAPI, Uvicorn ASGI server
Web dashboard	Node.js/npm, React 19, Vite 8
Mobile app	Node.js/npm, Expo SDK 57, EAS CLI
Firmware	Arduino IDE, ESP32 Arduino core (WiFi/WebServer/ESPmDNS)

Table 5 — Network Requirements

Link	Requirement
Client ↔ Backend	HTTPS (Render-terminated TLS) or HTTP (local dev)
Backend ↔ ESP32 (local)	Same LAN/Wi-Fi as the ESP32
Backend ↔ ESP32 (remote)	Persistent public tunnel (ngrok) bridging the backend to the ESP32's LAN
Client ↔ Supabase	HTTPS
Backend ↔ Gemini / Twilio	HTTPS (outbound only)

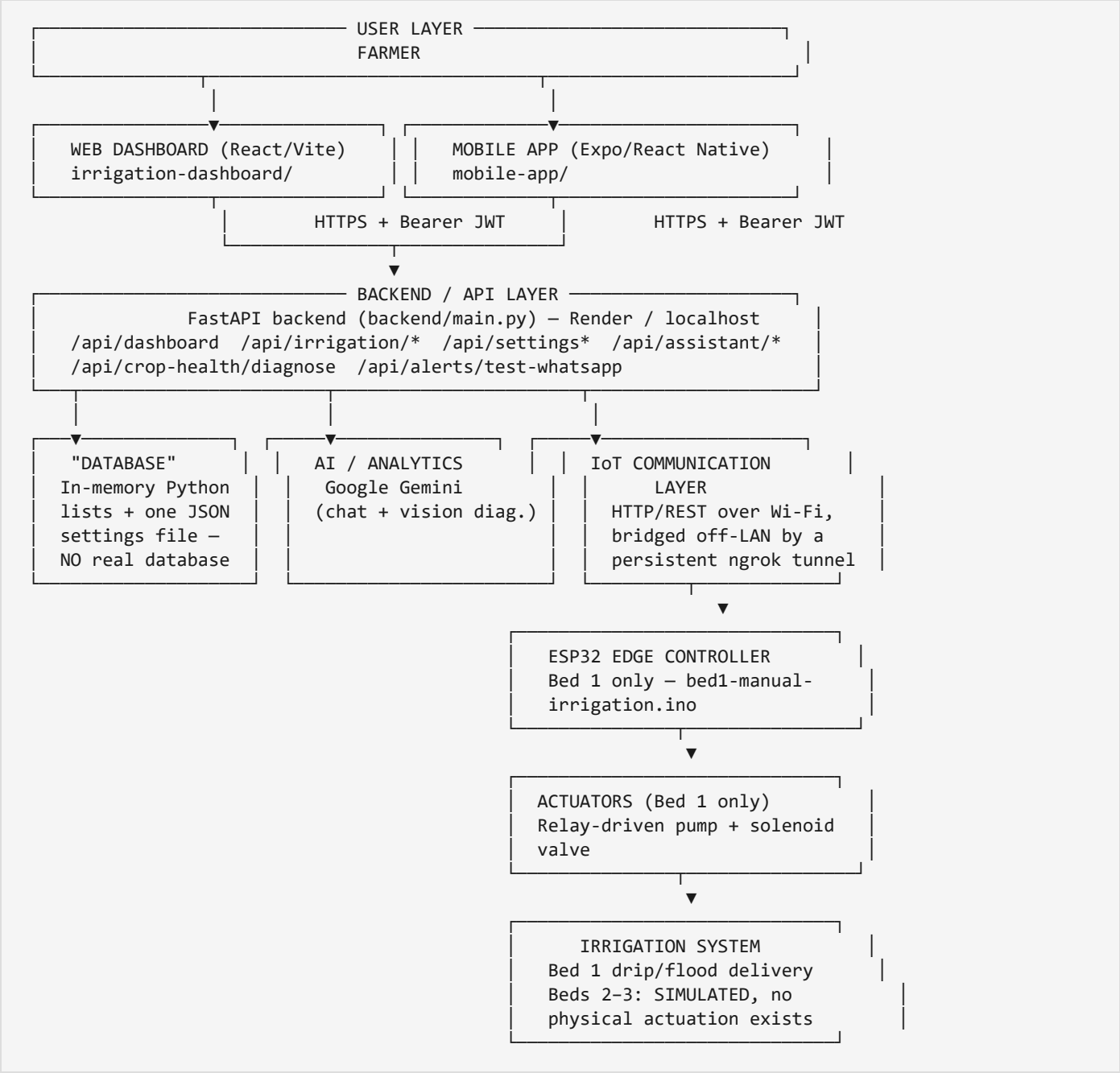
Table 6 — Security Requirements

ID	Requirement	Status
SR-1	Every API request authenticated	Implemented (Supabase JWT, verified via JWKS)
SR-2	Per-user authorization / data scoping	Not implemented — single shared demo farm, `sub` claim unused for scoping
SR-3	ESP32 endpoints authenticated	Implemented (static shared-secret query param)
SR-4	Secrets kept out of source control	Implemented (`.env`, Render env vars)
SR-5	Transport encryption on all public hops	Implemented for client↔backend↔Supabase (HTTPS); ESP32↔tunnel is HTTP under an HTTPS-terminated ngrok edge
SR-6	Rate limiting / abuse protection	Not implemented

---

4. Overall System Architecture

Figure 1.



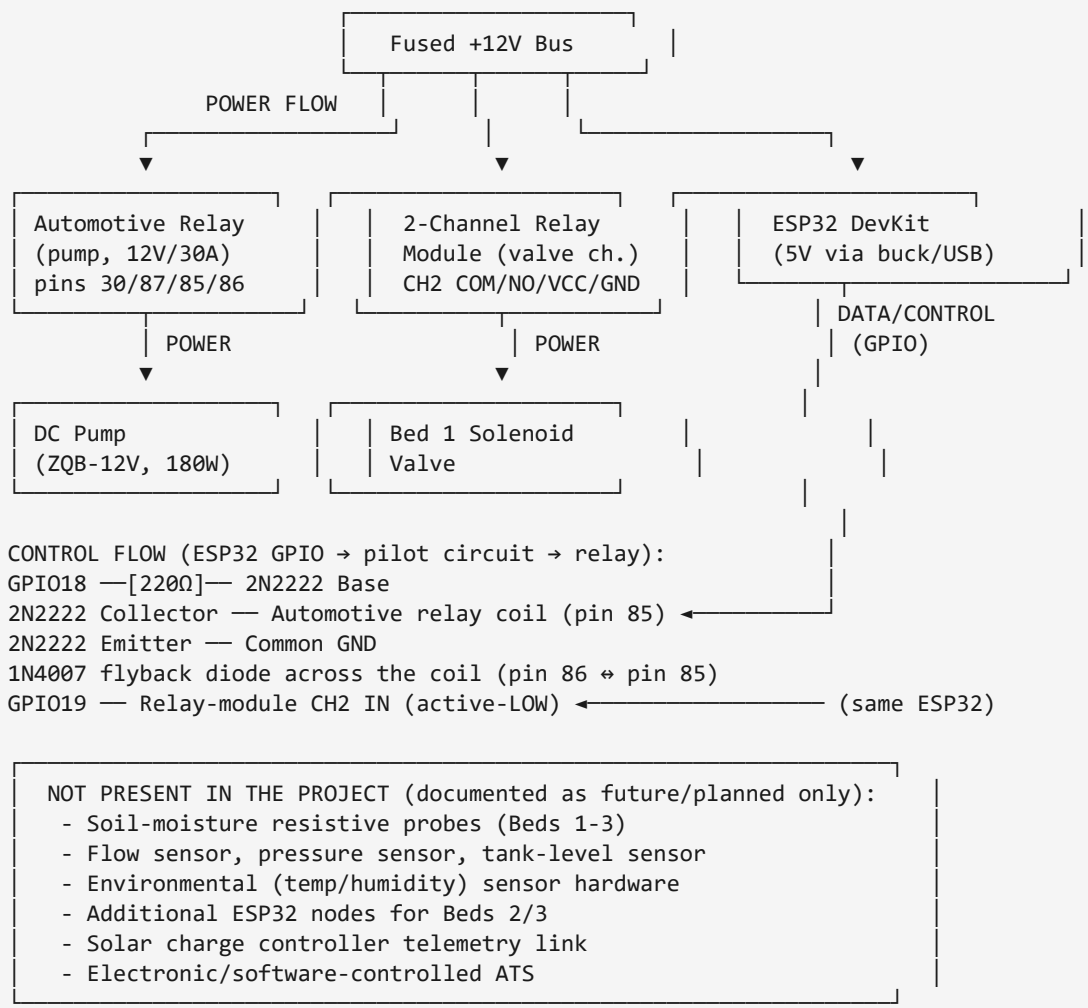
Energy infrastructure (documented, not software-integrated):

```
SOLAR PANEL (50W) → CHARGE CONTROLLER → BATTERY (12V) → POWER
MANAGEMENT (none — no controller/monitoring hardware wired to any software)
→ ATS (manual demo: physically disconnect/reconnect a wire; not an
electronic/automatic switch, not software-monitored) → PUMP LOAD (Bed 1's
12V pump, actually powered from a fused 12V bus, not confirmed to be the
solar/battery bus specifically in the current bench wiring)
```

No line in the diagram above between "energy infrastructure" and "backend" represents a real data connection — the dashboard's Energy Management page is entirely simulated (`src/lib/energySimulation.js`), not fed by the solar hardware. This is stated explicitly in that file's own header comment.

5. Hardware Architecture

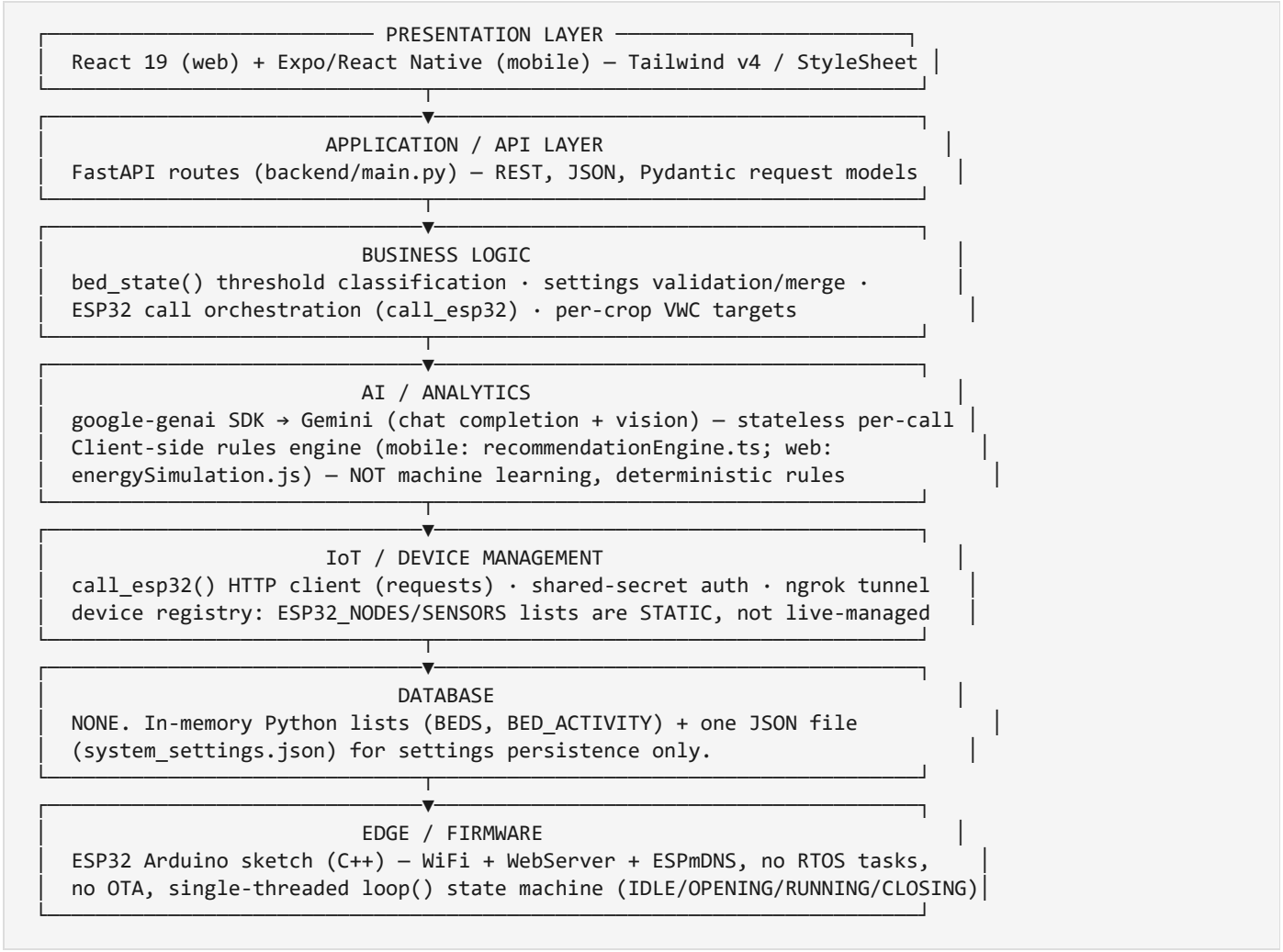
Figure 2.



Power flow is the fused 12V bus feeding both relay-controlled loads. Control flow is entirely ESP32 GPIO-driven (two channels: GPIO18 for the pump's transistor pilot, GPIO19 for the valve's relay-module channel). No data flows *from* any sensor into the ESP32 in the current build — this node is output-only (actuation), not a sensing node, despite the "sensor-node" directory name it lives in (a naming holdover from the original combined sensor+irrigation node plan in `PROJECT-OBJECTIVES.md`).

6. Software Architecture

Figure 3.


PRESENTATION LAYER
React 19 (web) + Expo/React Native (mobile) — Tailwind v4 / StyleSheet
APPLICATION / API LAYER
FastAPI routes (backend/main.py) — REST, JSON, Pydantic request models
BUSINESS LOGIC
bed_state() threshold classification · settings validation/merge ·
ESP32 call orchestration (call_esp32) · per-crop VWC targets
AI / ANALYTICS
google-genai SDK → Gemini (chat completion + vision) — stateless per-call
Client-side rules engine (mobile: recommendationEngine.ts; web:
energySimulation.js) — NOT machine learning, deterministic rules
IoT / DEVICE MANAGEMENT
call_esp32() HTTP client (requests) · shared-secret auth · ngrok tunnel
device registry: ESP32_NODES/SENSORS lists are STATIC, not live-managed
DATABASE
NONE. In-memory Python lists (BEDS, BED_ACTIVITY) + one JSON file
(system_settings.json) for settings persistence only.
EDGE / FIRMWARE
ESP32 Arduino sketch (C++) — WiFi + WebServer + ESPmDNS, no RTOS tasks,
no OTA, single-threaded loop() state machine (IDLE/OPENING/RUNNING/CLOSING)

Concern	Technology	Status
Frontend framework	React 19 (web), Expo/React Native 0.86 (mobile)	Implemented
Backend framework	FastAPI + Uvicorn	Implemented
Database	—	Not implemented (in-memory + one JSON file)
API technology	REST/JSON over HTTP(S)	Implemented
IoT protocol	HTTP/REST (no MQTT, no CoAP, no Modbus)	Implemented (HTTP only)
Firmware	Arduino C++ on ESP32	Implemented (Bed 1 only)
Authentication	Supabase (GoTrue) JWT, verified via JWKS	Implemented
AI/ML	Google Gemini (hosted, no local model)	Implemented (chat + vision)
Reporting	Static JSON payload (`dataReports` in `/api/dashboard`)	Implemented as mock data, not computed from real history
Notification/alert	Twilio WhatsApp	Implemented for irrigation events; not for fault/threshold events
GIS	Leaflet + React-Leaflet + Esri World Imagery tiles	Implemented (static farm boundary/markers)
Device management	—	Not implemented (no device registry, no OTA, no heartbeats)

---

7. Communication Architecture

Figure 4.

```
TELEMETRY PATH (currently: control-acknowledgement only, no sensor telemetry exists)
[ Bed 1 ESP32 ] --HTTP GET response--> [ ngrok tunnel / LAN ] --HTTPS/HTTP--> [ Backend ]
     --HTTPS + JWT--> [ Web Dashboard / Mobile App ]

CONTROL PATH (the path that actually matters in this system)
[ Dashboard/Mobile ] --HTTPS POST + Bearer JWT--> [ Backend /api/irrigation/start/1 ]
     --HTTP GET /irrigate?sec=N&key=SECRET--> [ ngrok tunnel ] --LAN HTTP-->
     [ ESP32 ] --GPIO--> [ Relay/Transistor ] --12V switching--> [ Pump / Valve ]
```

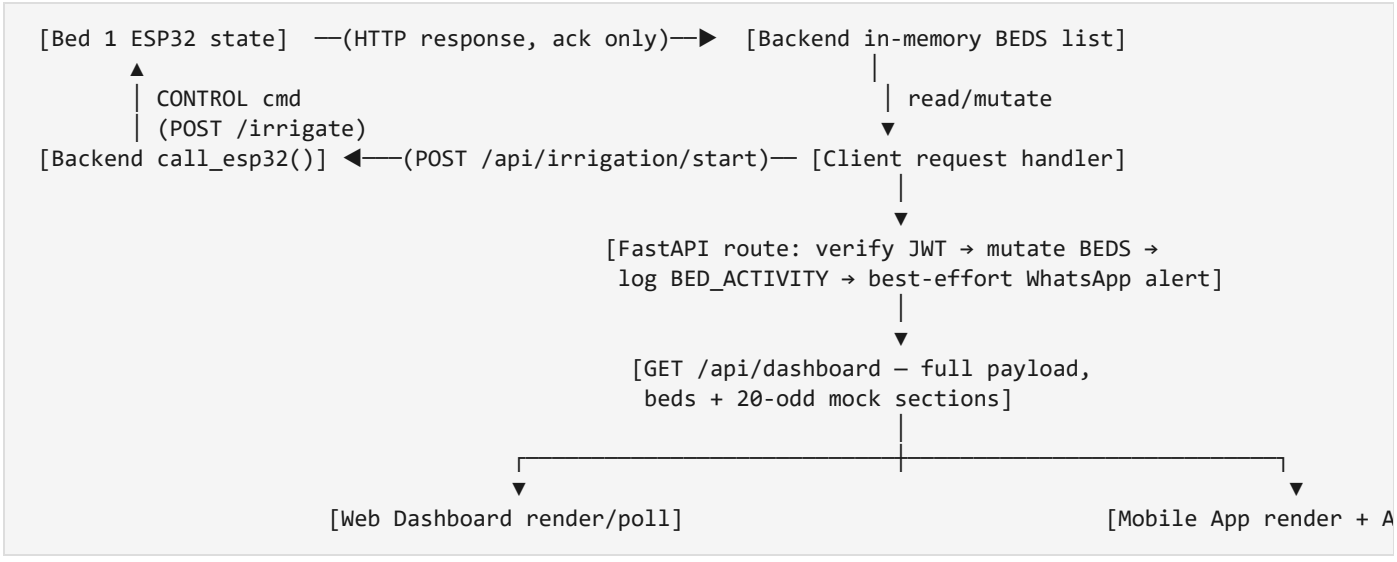
Protocol	Used?	Where
Wi-Fi	Yes	ESP32 ↔ home/farm router
HTTP/REST	Yes	Backend ↔ ESP32 (unencrypted on the LAN hop; the public ngrok edge itself terminates HTTPS from the internet side)
HTTPS	Yes	Client ↔ Backend, Client/Backend ↔ Supabase, Backend ↔ Gemini/Twilio
MQTT	**No** — despite being named as a future milestone in `PROJECT-OBJECTIVES.md`, it is not implemented anywhere	—
WebSockets	**No** — `socket.io-client` is an installed dependency in the web dashboard but has no confirmed live socket connection wired up	—
GSM/4G	No	—
Ethernet	No	—
Serial/Modbus	No	—

The ESP32's endpoints (`/irrigate`, `/stop`, `/status`) accept **only** HTTP GET with a `key` query parameter matched against a static shared secret (`ESP32_SHARED_SECRET`). There is no TLS termination on the ESP32 itself — the ngrok tunnel provides the public-facing HTTPS edge, but the LAN hop between the tunnel client and the ESP32 remains plain HTTP.

---

8. Data Flow Diagram

Figure 5.

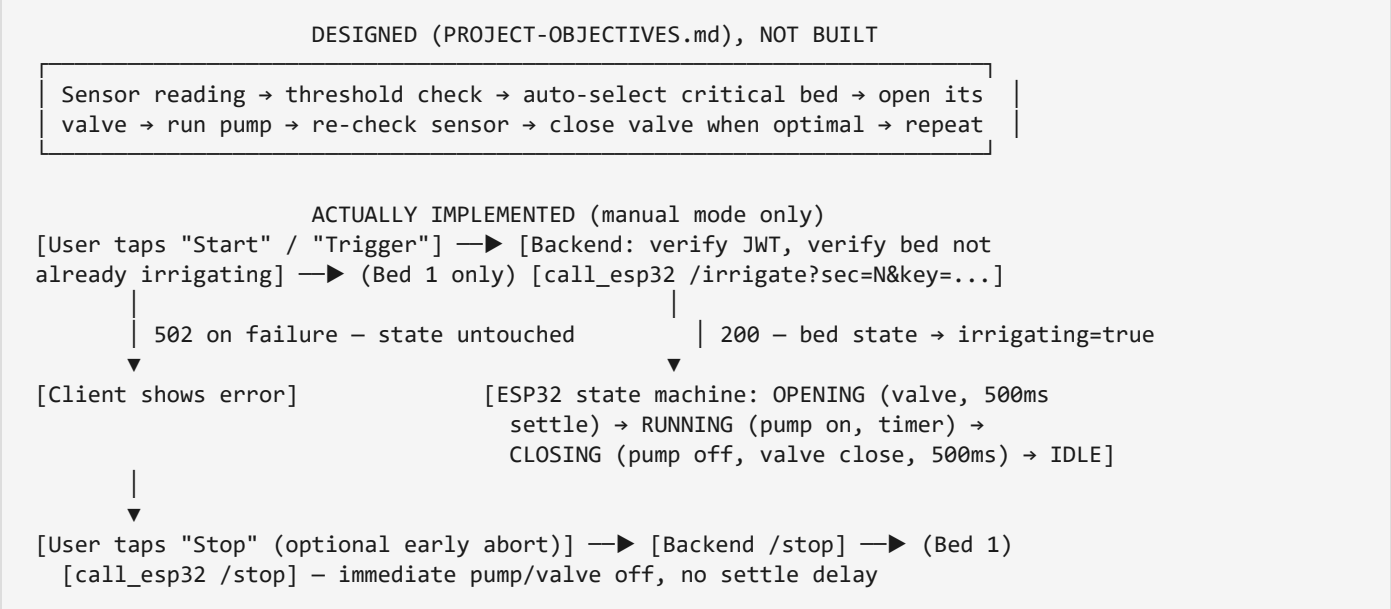


There is **no** background/scheduled data pipeline — no cron job, no sensor-polling loop, no rule engine evaluating thresholds continuously. Every number in the dashboard payload except `BEDS/BED_ACTIVITY` is a static literal returned unchanged on every call; only bed VWC and activity log entries are ever mutated, and only by direct client action (trigger/start/stop), never by a background process. "Sensor acquisition → edge processing → AI/analytics → decision engine" as a continuous automatic pipeline (per `PROJECT-OBJECTIVES.md`'s design) **does not exist in code**; the design is locked but unbuilt.

---

9. Irrigation Control Architecture

Figure 6.



- **Automatic mode**: exists as a UI toggle/label (`mode: "automatic"` in settings and the dashboard's System Mode control) but has **no corresponding backend behavior** — selecting it does not cause any autonomous action; it currently only *disables* the manual trigger button in the UI.
- **Manual mode**: fully implemented for Bed 1 (real hardware) and Beds 2/3 (simulated instant/ramped moisture bump, no physical actuation).
- **Manual override / Stop**: implemented — `/api/irrigation/stop/{bed_id}` is an early-abort path independent of the ESP32's own timer.
- **Safety conditions**: `MAX_IRRIGATION_MINUTES = 120` caps a single manual cycle server-side; the firmware separately caps at `MAX_RUN_SEC = 7200` (120 min) as a hardware-side backstop even if a malformed request slipped past the backend's own cap.
- **Fault detection**: the only fault path implemented is an unreachable ESP32 (`requests.RequestException` → `502`, state left untouched). There is **no** sensor-failure handling (no sensors exist to fail), **no** flow/ pressure anomaly detection, and **no** dry-run/leak protection logic, despite `water.leakDetectionEnabled` / `water.dryRunProtectionEnabled` existing as settings toggles with no code behind them.
- **Communication-failure handling**: implemented at the backend↔ESP32 hop only (the `call_esp32` HTTPException path above). No handling exists for a client losing connectivity mid-cycle — this is by

design, since the ESP32 owns its own timing once triggered and does not depend on the client staying connected.

---

10. Energy Management Architecture

Figure 7.

```
DOCUMENTED HARDWARE (PROJECT-OBJECTIVES.md, no software link):
[50W Solar Panel] → [Charge Controller] → [12V Battery]
       → [Power Management: none built] → [ATS: manual physical demo only]
       → [Pump Load]

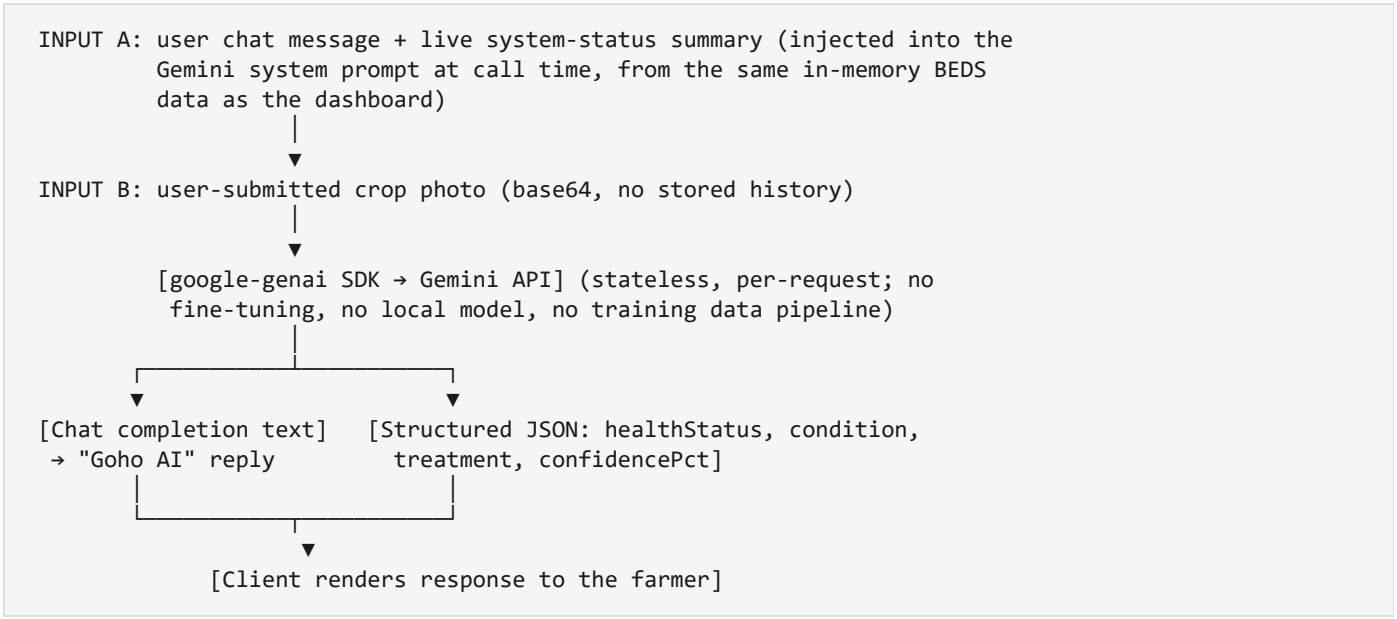
ACTUAL SOFTWARE (irrigation-dashboard/src/lib/energySimulation.js):
[Weather condition (mock/live-fetched icon)] → SOLAR_SCORE_BY_ICON lookup
       table → [solarScore 0-100]
[Time-of-day sine wave] → simulateGridStability() → [gridScore 0-100]
[Static battery % from backend payload] → [batteryPct]
       │
       ▼
recommendSource({solarScore, gridScore, batteryPct}) → { source: solar |
  battery | grid, reason: <human-readable string> }
       │
       ▼
[Energy Management page: Smart Power Recommendation card, ATS changeover
 diagram (GridChangeoverSimulation.jsx) — both read this one function so
 they never disagree with each other, but neither reflects real hardware]
```

How energy availability influences irrigation, in the actual system: it does **not**. `recommendSource()` and the irrigation-trigger endpoints are entirely independent code paths — nothing in the backend checks battery/solar state before allowing a pump to run. The relationship described in `PROJECT-OBJECTIVES.md` (irrigation scheduling aware of energy availability) is a design intent with no corresponding implementation.

---

11. AI Architecture

Figure 8.



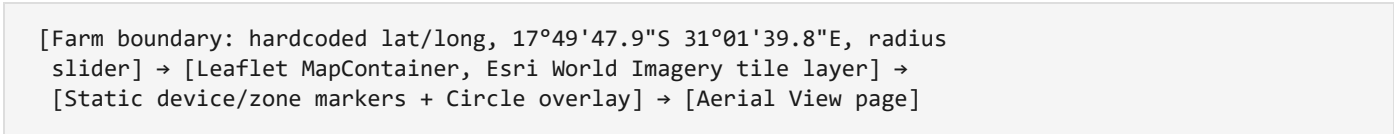
Category	Example	Where it lives
AI prediction (Gemini)	Chat reply, crop diagnosis	Backend `/api/assistant/chat`, `/api/crop-health/diagnose`
Rule-based automation	`bed_state()` threshold classification; `recommendSource()` energy heuristic; mobile `recommendationEngine.ts`	Backend + both clients, deterministic, no ML
Human decision	Every irrigation trigger/stop; every settings change	Client UI action

There is **no** AI/ML model trained on this project's own sensor history — there is no sensor history to train on. "IoT sensor data → preprocessing → model → prediction" as a pipeline (per the requested architecture) does not exist; the only AI calls are on-demand, user-initiated (chat message or photo upload), grounded by a snapshot of current mock state, not a learned model.

---

12. GIS and Network Monitoring

Figure 10.



- **Implemented**: satellite-imagery farm map, adjustable radius circle, marker/popup UI (`irrigation-dashboard/src/pages/AerialView.jsx`).
- **Not implemented**: no live device geolocation (marker positions are not derived from any ESP32's actual GPS/location), no connectivity/RSSI feed driving the map (the `ESP32_NODES` list with `rssiDbm`/`lastComm` fields exists in the backend but is static, not wired to the map view), no per-zone pressure/flow status (no such sensors exist to report).

- A Google Earth Engine upgrade (NDVI, change-over-time) is scoped in project notes but requires a GCP project/service account and a new backend endpoint — not started.

---

13. Alert and Notification Architecture

Figure 9.

```
                      IMPLEMENTED PATH
[Irrigation trigger/start/stop action] → [send_whatsapp_alert(message)]
       → [Twilio WhatsApp Business API] → [Farmer's WhatsApp number]
  (best-effort: TwilioRestException caught and logged, never raised to the
   caller — a failed alert never blocks the irrigation action)

                      NOT IMPLEMENTED (config exists, logic doesn't)
[Low water / low pressure / abnormal flow / pump failure / sensor failure /
 battery problem / solar problem / comms failure / irrigation anomaly]
       → NO EVENT ENGINE EXISTS to detect any of these
       → settings.notifications.events.* toggles exist and are user-editable,
         but nothing in the codebase reads them to decide whether to fire
```

Channel	Wired?
WhatsApp (Twilio)	Yes — the only implemented channel
SMS	No — toggle exists in settings, no code
Email	No — toggle exists in settings, no code
In-app	Partially — the dashboard's `alerts` array in `/api/dashboard` is static mock data, not generated from real events

---

14. API Architecture

Figure 12.

```
                      FastAPI app (backend/main.py)
┌──────────────────────────────────────────────────────────────────────┐
│  Every route below (except GET /) requires:                          │
│  Authorization: Bearer <Supabase JWT>  →  require_user() dependency  │
│  → PyJWKClient verifies signature against Supabase's public JWKS     │
└──────────────────────────────┬───────────────────────────────────────┘
                               ▼
GET  /                         — health/root, no auth
GET  /api/dashboard             — full payload (beds, KPIs, energy, reports…)
POST /api/irrigation/trigger/{id}— instant demo moisture bump + WhatsApp alert
POST /api/irrigation/start/{id}  — mobile/dashboard manual control; hits real
                                   ESP32 for bed_id==1 if ESP32_BASE_URL set
POST /api/irrigation/stop/{id}   — early-abort + moisture bump on completion
GET  /api/settings               — full editable config + device/sensor status
PUT  /api/settings               — merge + validate + persist to JSON file
POST /api/settings/maintenance   — simulated maintenance actions (one real:
                                   factory-reset restores DEFAULT_SETTINGS)
POST /api/alerts/test-whatsapp   — manual Twilio credential test
POST /api/assistant/chat         — Gemini chat, grounded with live BEDS snapshot
POST /api/crop-health/diagnose   — Gemini vision, photo → structured diagnosis
```

Request/response shape: JSON in, JSON out throughout; request bodies are Pydantic models (`StartIrrigationRequest`, `SettingsUpdate`, `MaintenanceRequest`, `ChatRequest`, `CropHealthRequest`) giving automatic validation and a generated OpenAPI schema at `/openapi.json`. Errors are standard FastAPI `HTTPException` JSON (`{"detail": "..."}`) with conventional status codes (401 unauthenticated, 404 unknown bed, 409 already-irrigating, 502 hardware unreachable, 503 JWKS unreachable).

Device communication: one-directional from backend to device (`call_esp32()`, plain `requests.get`). The ESP32 never calls the backend — there is no device-initiated telemetry push, no webhook, no polling loop on the ESP32 side.

---

15. Database Architecture

There is no database in this project. This section documents the **data model that would need one**, not an existing schema.

Figure 12 — Entity Diagram (conceptual; not yet backed by any storage engine)

```
USER (Supabase-managed; email, id — the ONLY entity in a real database
today, and it's Supabase's, not this project's own schema)
   │ 1
   │
   │ N (not yet modeled — every farm today is the one shared demo farm)
   ▼
BED  (id, name, crop, vwc, state, vwcTrigger, vwcOptimalMin,
      vwcOptimalMax, lastIrrigation, durationMin, waterUsedL, soilTempC,
      trendVwc[], airTempC, humidityPct, pumpStatus, valveStatus,
      irrigating)                — Table 7 below
   │ 1
   │
   │ N
   ▼
BED_ACTIVITY (id, time, title, subtitle, color)  — an append-only log,
                                                    capped by nothing;
                                                    grows unbounded
                                                    in-process

SETTINGS (singleton, not per-user) — persisted to system_settings.json,
          the only entity with disk persistence at all

[NOT MODELED / NOT PERSISTED ANYWHERE: sensor readings, energy readings,
 AI predictions, reports — all of these are returned as static literals
 from GET /api/dashboard, not read from or written to any store]
```

Table 7 — Bed / Crop Data Model (as implemented, in-memory)

Field	Type	Source
id, name, crop	static	Seeded at process start
vwc	mutable	Changed only by trigger/start/stop endpoints
state	derived	`bed_state()`, recomputed from `vwc` vs. thresholds
vwcTrigger / vwcOptimalMin / vwcOptimalMax	static per-crop	`CROP_TARGETS`
pumpStatus / valveStatus / irrigating	mutable	Set by irrigation endpoints
trendVwc[]	mutable, capped at 9	Appended on each mutation

Gap: none of the above survives a process restart except `SETTINGS` (the one JSON file). `PROJECT-OBJECTIVES.md` and `PROJECT_CONTEXT.md` both independently confirm SQLite persistence was planned and never started.

---

16. Security Architecture

Control	Status	Detail
Authentication	Implemented	Supabase-issued JWT (ES256/RS256), verified against public JWKS, `authenticated` audience enforced
Authorization / RBAC	**Not implemented**	Any authenticated user has full access to the one shared demo farm; no roles, no per-user scoping (the JWT's `sub` claim is available but unused for this)
Device security	Weak	Static shared-secret query-string key on the ESP32; no TLS on the LAN hop; no key rotation
API security	Partial	JWT-gated, but no rate limiting, no request throttling, no input size limits beyond Pydantic type validation
Data encryption in transit	Implemented (client↔backend↔Supabase/Gemini/Twilio); **not** on the backend↔ESP32 LAN hop	—
Data encryption at rest	N/A	No persisted data beyond one non-sensitive settings JSON file
Credential management	Implemented	`.env` locally, Render environment variables in production; no secret ever shipped in a client bundle
CORS	Deliberately wildcard (`allow_origins=["*"]`)	Acceptable for a no-cookie, bearer-token API with no browser-credential surface, but broader than a production system would choose
Secure communication (ESP32)	Weak	Plain HTTP + static key; adequate only because the ngrok edge itself is HTTPS-terminated from the public side

Identified weaknesses and recommendations:

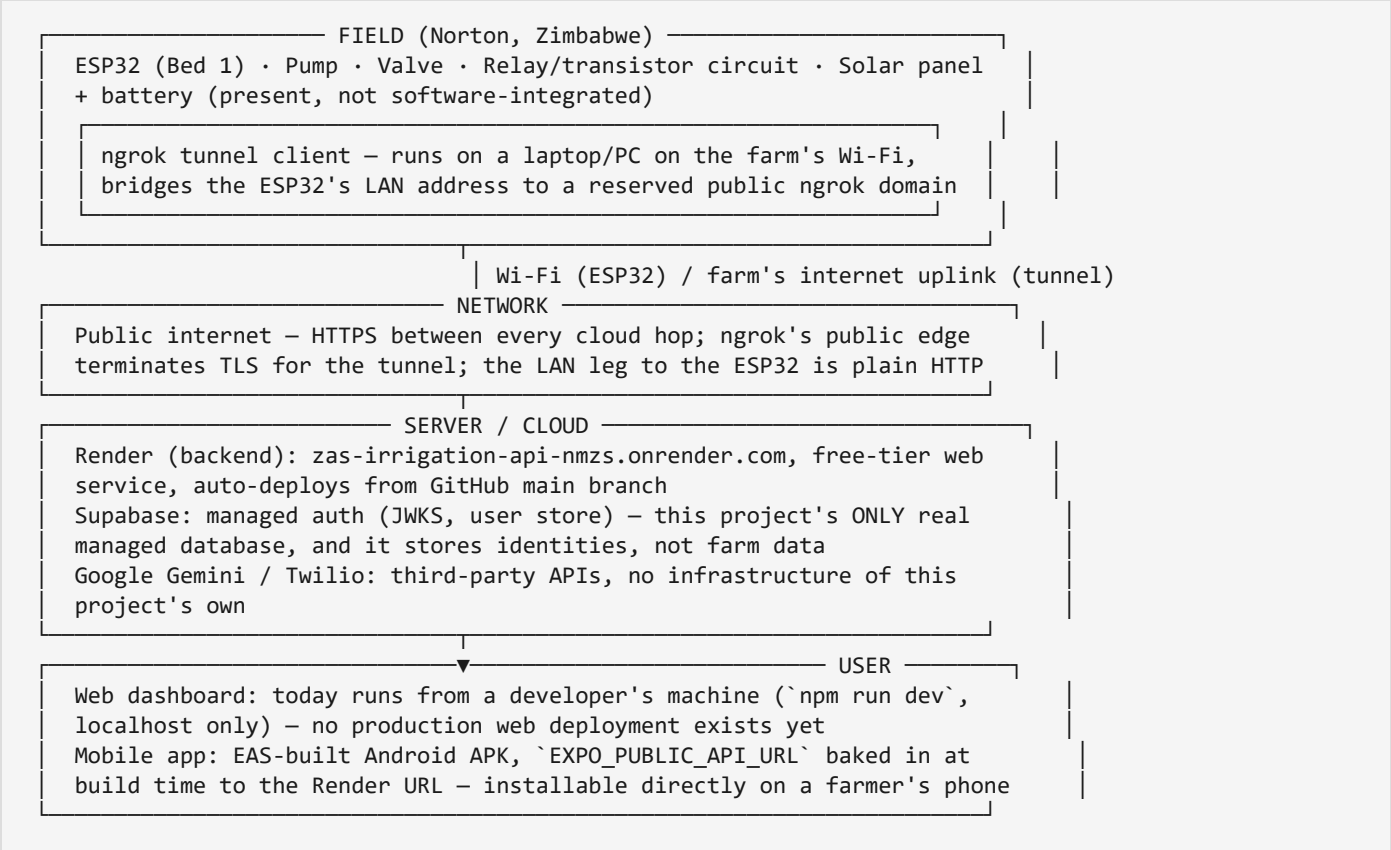
1. **No authorization model** — recommend scoping data per Supabase `sub` once multiple real farms/users exist; today it's a non-issue only because there is exactly one shared farm.
2. **Static ESP32 shared secret, sent as a URL query parameter** — visible in logs (ngrok's own request logs, browser history if ever pasted manually). Recommend moving it to a header, and rotating it periodically.
3. **No rate limiting anywhere** — a compromised or buggy client could hammer `/api/irrigation/start/1` (bounded only by the `409 already-irrigating` check) or the Gemini-backed endpoints (bounded only by Gemini's own quota).
4. **No audit trail with user identity** — `BED_ACTIVITY` logs *what* happened, never *who* did it, despite the JWT containing that information on every call.

5. **Single point of physical failure for remote hardware access** — the ngrok tunnel depends on one machine at the farm staying powered on; no redundancy, no health-check/alert if it drops.

---

17. Deployment Architecture

Figure 13.



---

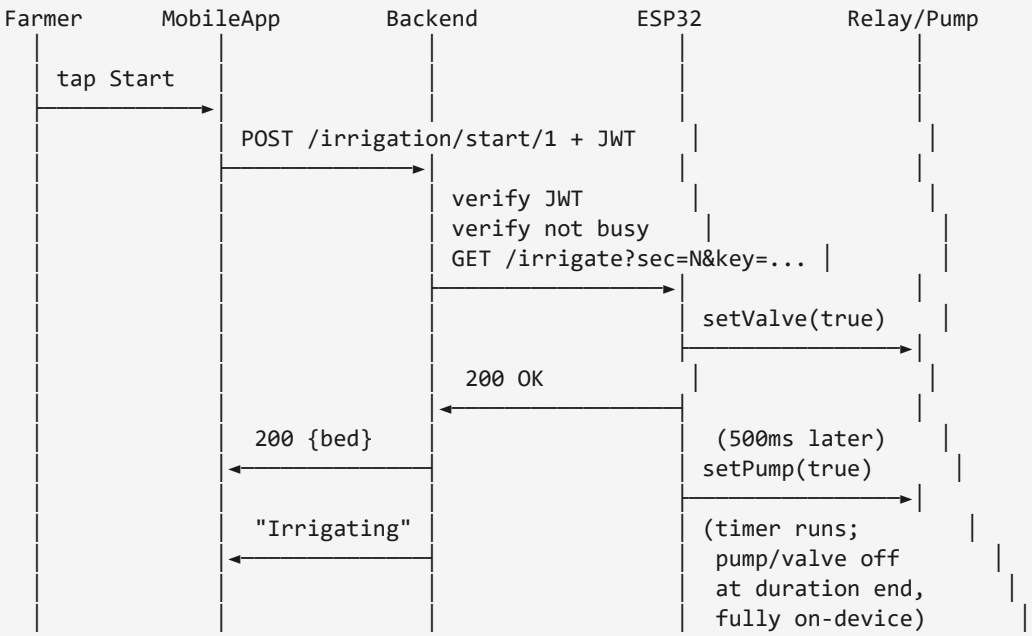
18. System Sequence Diagrams

UML-style, actor/lifeline notation rendered in monospace for document portability. * marks a step that is simulated rather than driven by real hardware/sensors.

A. Automatic Irrigation — NOT IMPLEMENTED

No automatic-mode sequence exists in code. Selecting "Automatic" in the UI only disables the manual trigger button; no background process evaluates thresholds or calls any irrigation endpoint on its own. This diagram is intentionally omitted rather than fabricated — see §23.

B. Manual Irrigation (Bed 1, real hardware)



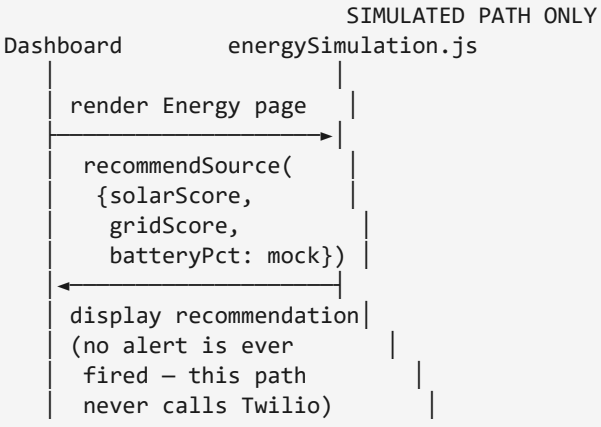
C. Sensor Data Transmission — NOT IMPLEMENTED

No sensor exists on any node in this project that reports a reading to the backend. The ESP32's only outbound traffic is an HTTP response acknowledging a control command, never an unsolicited telemetry push. Omitted for the same reason as (A) — see §23.

D. Low-Pressure Alert — NOT IMPLEMENTED

No pressure sensor exists; `water.pressureMinBar/pressureMaxBar` are settings values with no code path that ever reads a pressure value against them. Omitted — see §23.

E. Low-Battery/Energy Event

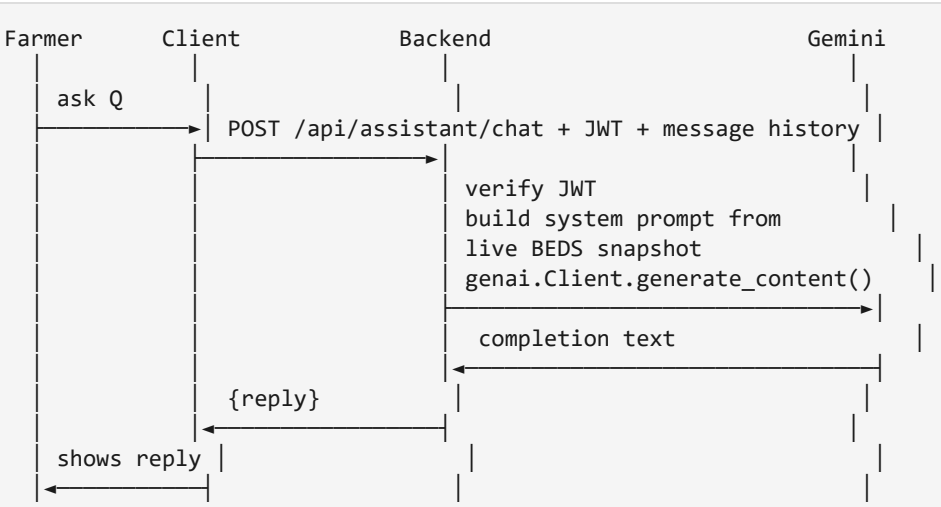


No real low-battery *event* triggers a notification anywhere in the backend; `energy.lowBatteryPct/criticalBatteryPct` settings exist unused by any alert logic.

F. Device Failure (ESP32 unreachable)



G. AI Recommendation (chat)



---

19. Technology Stack

Table 8 — Technology Stack

Component	Technology	Purpose	Where it runs	Communication
Web dashboard	React 19 + Vite 8 + Tailwind v4	Kiosk-oriented farm UI	Developer machine (`localhost:5173`); no production deploy yet	HTTPS/HTTP to backend
Mobile app	Expo SDK 57 / React Native 0.86 / TypeScript	Farmer-facing app, remote-capable	Farmer's Android phone (EAS-built APK)	HTTPS to backend
Backend API	FastAPI + Uvicorn (Python)	Single source of truth, orchestrates AI/alerts/hardware	Render (cloud) or `localhost:8021` (dev)	REST/JSON
Auth	Supabase (GoTrue)	Shared identity for both clients	Supabase cloud	HTTPS, JWT bearer
AI	Google Gemini (`google-genai`)	Chat assistant, crop diagnosis	Google Cloud (external)	HTTPS, backend-only
Alerts	Twilio WhatsApp Business API	Irrigation-event notifications	Twilio cloud (external)	HTTPS, backend-only
Hardware control	Arduino C++ on ESP32	Bed 1 pump/valve actuation	Physical ESP32 at the farm	HTTP over Wi-Fi/tunnel
Remote hardware bridge	ngrok (reserved static domain)	Exposes the LAN-only ESP32 publicly	A machine on the farm's Wi-Fi	HTTPS (public) / HTTP (LAN)
Hosting/CD	Render (Blueprint from `render.yaml`)	Backend deployment	Render cloud	Auto-deploy from GitHub `main`
Source control	Git / GitHub	Version control, deploy source	GitHub (`Bhamu101/zas-project`)	—
Mapping	Leaflet + React-Leaflet + Esri tiles	Farm boundary visualization	Web dashboard (client-side)	Tile HTTPS requests

---

20. Hardware/Software Interface Table

Table 9 — Hardware/Software Interface

Hardware Component	Interface	Protocol	Controller	Data/Command	Direction
2N2222 transistor (pump pilot)	GPIO18 → 220Ω resistor → base	Digital GPIO	ESP32	HIGH/LOW (pump on/off)	ESP32 → Transistor
2-channel relay module (valve)	GPIO19 → module IN2	Digital GPIO, active-LOW	ESP32	HIGH/LOW (valve open/closed)	ESP32 → Relay
Automotive relay (pump power)	Coil pins 85/86, contacts 30/87	12V switching	Transistor circuit	On/off pump power path	Transistor → Relay → Pump
DC pump	Relay contact 87 → pump (+)	12V DC	Automotive relay	Power only, no feedback	Relay → Pump
Solenoid valve	Relay-module CH2 NO	12V DC	Relay module	Power only, no feedback	Relay → Valve
ESP32 ↔ Backend	Wi-Fi → HTTP GET	HTTP/1.1, query-string auth	Backend `requests` client	`/irrigate?sec=N&key=...`, `/stop?key=...`, `/status?key=...`	Backend → ESP32 (request), ESP32 → Backend (ack only)
Backend ↔ Client	HTTPS REST	HTTP/1.1 over TLS, JSON	FastAPI	Full dashboard payload, irrigation commands	Bidirectional
Backend ↔ Supabase	HTTPS	JWKS fetch (verification only)	PyJWKClient	Public signing keys	Supabase → Backend (one-way, keys only)
Backend ↔ Gemini	HTTPS	REST (SDK-wrapped)	`google-genai`	Prompt in, completion/JSON out	Bidirectional
Backend ↔ Twilio	HTTPS	REST (SDK-wrapped)	`twilio` Python client	WhatsApp message payload	Backend → Twilio (one-way)

No entry exists for "Sensor → ESP32" because no sensor is wired to any ESP32 in this project.

---

21. Testing and Validation

- **Frontend**: `npx eslint` (both web and mobile) run clean before every change is considered done; `npx vite build` / `npx tsc --noEmit` used as a compile-correctness gate.
- **Backend**: no automated test suite exists (`backend/` has no `tests/` directory or `pytest` dependency) — verification has been manual, via direct `curl` calls against running endpoints during development sessions

(documented throughout `CHANGED_FILES.txt`).

- **Firmware**: verified via Serial Monitor logging and a documented 4-step bench-test sequence (pilot circuit alone → add valve → add pump in a bucket → real bed), not via any automated test harness (not practical for embedded C++ in this toolchain without additional infrastructure).
- **End-to-end**: verified manually per feature, via direct HTTP calls and live UI interaction — no CI pipeline, no automated end-to-end test suite.

---

22. Diagram Index

Individual high-resolution diagram files, generated alongside this document, in `architecture-diagrams/`:

File	Content	Source Figure
`01-overall-system-architecture.png`	Overall system, layered	Figure 1
`02-hardware-architecture.png`	Hardware, power/control flow	Figure 2
`03-software-architecture.png`	Software layers	Figure 3
`04-communication-architecture.png`	Protocols, telemetry/control paths	Figure 4
`05-data-flow.png`	Data flow diagram	Figure 5
`06-irrigation-control.png`	Manual control sequence + gaps	Figure 6
`07-energy-management.png`	Simulated energy logic	Figure 7
`08-ai-architecture.png`	Gemini-backed AI flow	Figure 8
`09-alert-architecture.png`	WhatsApp path + missing paths	Figure 9
`10-gis-network.png`	Farm map / device status	Figure 10
`11-database-er.png`	In-memory data model (no DB)	Figure 11
`12-api-architecture.png`	API route map	Figure 12
`13-deployment-architecture.png`	Field/Network/Cloud/User	Figure 13
`14-security-architecture.png`	Auth, gaps, recommendations	Figure 14

---

23. Architecture Validation and Implementation Gaps

Table 10 — Implementation Status Matrix

Subsystem	Status
Web dashboard UI (9 pages)	**Implemented** (against mock/in-memory backend)
Mobile app UI	**Implemented** (against the real backend)
Backend REST API (11 routes)	**Implemented**
Supabase authentication	**Implemented**
Bed 1 pump/valve control (real hardware)	**Implemented**
Remote (off-LAN) control of Bed 1	**Implemented** (Render + ngrok + shared secret)
AI chat assistant	**Implemented**
AI crop-photo diagnosis	**Implemented**
WhatsApp alerts on irrigation actions	**Implemented**
Settings persistence (JSON file)	**Implemented**
Farm map (GIS)	**Partially implemented** — static map, no live device data
Automatic (threshold-driven) irrigation	**Planned** — design locked, zero code
Soil-moisture sensors (Beds 1–3)	**Missing** — no probes wired
Flow / pressure / tank / climate sensors	**Missing** — settings reference thresholds for these with no sensors behind them
Additional ESP32 nodes (Beds 2/3)	**Missing**
Energy hardware telemetry (solar/battery/ATS)	**Missing** — dashboard figures are 100% simulated
Electronic/automatic ATS	**Missing** — current demo is a manual physical disconnect
Fault/threshold-based alerting (non-WhatsApp channels, sensor faults)	**Missing** — toggles exist, no detection logic
Database / persistent history	**Missing** — no SQLite, no time-series store
MQTT	**Missing** — HTTP/REST used instead throughout
Device management (heartbeats, OTA)	**Missing** — device lists are static demo data
Authorization/RBAC	**Missing** — authentication only, single shared farm
Automated test suite (backend/firmware)	**Missing**

Recommended immediate priorities (not requested to be exhaustive, but flagged because they block the largest gap between documented design and built system): 1. Wire at least one real soil-moisture probe to close the loop from "sensor reading" to "automatic decision" — currently the single largest gap between `PROJECT-OBJECTIVES.md`'s stated design and the running system. 2. Add a minimal persistence layer (even SQLite, as originally planned) so bed/activity state survives a restart — currently a hard restart silently erases the demo's entire irrigation history.

---

24. Scalability

The current architecture scales poorly beyond a single demo farm by design, not oversight — it was built for a 14-day exhibition push, not multi-tenant production use:

- **Backend state**: a single Python process's in-memory lists cannot serve more than one farm without a real per-farm data model and a real database.
- **ESP32 fleet**: `ESP32_NODES`/`SENSORS` being static demo lists means there is currently no mechanism to add a real second node without editing backend source code.
- **Render free tier**: cold-starts after inactivity (~30-60s), and has no autoscaling — acceptable for a demo, not for concurrent multi-farm load.
- **ngrok free static domain**: one tunnel, one machine, one point of failure — does not scale to multiple field sites without one tunnel (and one always-on machine) per site.

25. Risks and Limitations

- **Single point of failure for remote hardware control**: the ngrok tunnel machine going offline silently degrades Bed 1's real hardware to unreachable (`502`) while the rest of the app keeps working — easy to miss.
- **Data loss on backend restart**: every bed reading, activity log entry, and irrigation history resets to seed values on any Render redeploy or crash.
- **No sensor validation**: because no real sensors feed the system except Bed 1's actuator acknowledgement, there is no mechanism today to detect drift between displayed (simulated) data and physical reality for Beds 2/3.
- **Static shared secret**: the ESP32's only access control is a single string that, if leaked, grants full pump control with no way to detect misuse (no logging of caller identity at the firmware layer).

26. Future Improvements

1. Build the automatic irrigation control loop described in `PROJECT-OBJECTIVES.md` §1 once real sensors are wired.
2. Add SQLite (or equivalent) persistence for bed/activity/settings history.
3. Wire real energy telemetry (solar panel, battery, charge controller) to replace the simulated Energy Management page.
4. Implement the fault-detection logic behind the already-existing notification-settings toggles.
5. Add authorization/RBAC once more than one real farm/user exists.
6. Replace the ESP32's static shared-secret with a rotating token or per-device credential.
7. Add an automated backend test suite.

27. Conclusion

This project has successfully built a working, remotely-controllable irrigation control path from a farmer's phone to a physical pump and valve — a genuinely complete vertical slice of the intended system, end to end, including real cloud deployment, real authentication, and real AI features. What it has **not** yet built is the automatic, sensor-driven decision loop that is the system's core stated purpose, nor the energy-aware layer described in the project's objectives. Both remain locked-in designs, not implementations. This document exists so that gap is visible and precisely scoped, not so it can be papered over.

---

28. Executive Architecture Summary

The Smart Energy-Driven Irrigation system, as it exists in code today, is a **working manual-control and monitoring platform with one real hardware node**, wrapped in a considerably more complete-looking pair of client applications. A farmer signs in (via Supabase) on either a web dashboard or a mobile app, both of which poll a single FastAPI backend for farm status and post commands back to it. That backend is deployed on Render, publicly reachable over HTTPS, and is the only component that holds credentials for three external services: Google Gemini (a conversational assistant and a photo-based crop diagnosis feature), Twilio (WhatsApp alerts fired whenever an irrigation cycle starts or stops), and one physical ESP32 microcontroller wired to Bed 1's water pump and solenoid valve. Because that ESP32 lives on a home Wi-Fi network with no public IP, a persistent ngrok tunnel — running on a machine at the farm — bridges it to the internet, and every request to it is gated by a static shared secret so a public tunnel doesn't become an open invitation to run someone's water pump.

That real hardware path is genuinely complete: tapping "Start" on the mobile app, from anywhere with internet access, authenticates the request, calls the ESP32 through the tunnel, and the ESP32 itself — independent of whether the phone stays connected — runs a timed valve-open, pump-on, pump-off, valve-close sequence before returning to idle. If the ESP32 or the tunnel is unreachable, the backend fails loudly (`502`) rather than lying about irrigation having started, which matters more in a system that moves water than it would in most software.

Everything else the dashboards display — the other two beds' moisture levels, every energy and solar figure, flow/pressure/tank sensor readings, seasonal reports, the farm's device list — is realistic-looking but entirely generated by the backend on every request, not read from any sensor, database, or historical record. There is no database in this project at all; the only thing that survives a server restart is one settings file. There is no automatic irrigation: the system's stated core purpose — reading soil moisture and deciding, on its own, which bed needs water — has a fully worked-out design in the project's own planning documents but zero lines of implementing code. The same is true of any electronic energy-management logic tying solar/battery state to irrigation scheduling — a locked design decision, not a built system.

In short: this is a real, working, remotely-operable irrigation *actuation* system with a genuinely deployed cloud backend and real AI features, built around a single proof-of-concept hardware node — not yet the fully autonomous, sensor-driven, energy-aware system the project's objectives describe, and this document is written to make that distinction impossible to miss.